

PRACTICAL 28 -

Objective - Basic operations on Double Linked list

1. **Create a Doubly Linked List**

Write a program to create a doubly linked list with `n` nodes and display it in forward order.

2. **Insert at Beginning**

Implement a function to insert a node at the beginning of a doubly linked list.

3. **Insert at End**

Write a program to insert a node at the end of a doubly linked list.

4. **Insert at a Specific Position**

Add a node at a given position in the doubly linked list.

5. **Delete from Beginning**

Remove the first node from the doubly linked list.

6. **Delete from End**

Remove the last node from the doubly linked list.

7. **Delete from a Given Position**

Delete a node from a specific position in a doubly linked list.

8. **Traverse in Forward Direction**

Display all elements from head to tail.

9. **Traverse in Backward Direction**

Display all elements from tail to head using `prev` pointers.

10. **Count Nodes in a Doubly Linked List**

Return the total number of nodes in a doubly linked list.

11. **Search an Element in Doubly Linked List**

Search for a value in a doubly linked list and return its position.

12. **Reverse a Doubly Linked List**

Reverse a doubly linked list using pointer manipulation.

13. **Insert Before a Given Node**

Insert a new node before a specified node in a doubly linked list.

14. **Insert After a Given Node**

Insert a new node after a specified node in a doubly linked list.

15. **Delete a Node by Value**

Find a node by its value and delete it from the list.

16. Sort a Doubly Linked List using Bubble Sort

Implement bubble sort to sort the data in the doubly linked list.

CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Definition of a node in the doubly linked list
5  typedef struct Node {
6      int data;
7      struct Node* prev;
8      struct Node* next;
9  } Node;
10
11 // Function to create a new node
12 Node* create_node(int data) {
13     Node* new_node = (Node*)malloc(sizeof(Node));
14     new_node->data = data;
15     new_node->prev = NULL;
16     new_node->next = NULL;
17     return new_node;
18 }
19
20 // Create a doubly linked list with n nodes and display it
21 Node* create_list(int n) {
22     Node* head = NULL;
23     Node* tail = NULL;
24     for (int i = 0; i < n; i++) {
25         int data;
26         printf("Enter data for node %d: ", i + 1);
27         scanf("%d", &data);
28         Node* new_node = create_node(data);
29         if (!head) {
30             head = new_node;
31         } else {
32             tail->next = new_node;
33             new_node->prev = tail;
34         }
35         tail = new_node;
36     }
37     return head;
38 }
39
40 // Display list in forward direction
41 void display_forward(Node* head) {
42     Node* temp = head;
43     while (temp) {
44         printf("%d -> ", temp->data);
```

```
45     temp = temp->next;
46 }
47 printf("NULL\n");
48 }
49
50 // Display list in backward direction
51 void display_backward(Node* tail) {
52     while (tail) {
53         printf("%d -> ", tail->data);
54         tail = tail->prev;
55     }
56     printf("NULL\n");
57 }
58
59 // Insert at the beginning
60 Node* insert_at_beginning(Node* head, int data) {
61     Node* new_node = create_node(data);
62     if (head) {
63         head->prev = new_node;
64         new_node->next = head;
65     }
66     return new_node;
67 }
68
69 // Insert at the end
70 Node* insert_at_end(Node* head, int data) {
71     Node* new_node = create_node(data);
72     if (!head) return new_node;
73     Node* temp = head;
74     while (temp->next) {
75         temp = temp->next;
76     }
77     temp->next = new_node;
78     new_node->prev = temp;
79     return head;
80 }
81
82 // Insert at a specific position
83 Node* insert_at_position(Node* head, int data, int position) {
84     if (position == 1) return insert_at_beginning(head, data);
85     Node* new_node = create_node(data);
86     Node* temp = head;
87     for (int i = 1; i < position - 1 && temp; i++) {
88         temp = temp->next;
89     }
90     if (!temp) return head;
91     new_node->next = temp->next;
92     if (temp->next) temp->next->prev = new_node;
93     temp->next = new_node;
94     new_node->prev = temp;
95     return head;
```

```
96 }
97
98 // Delete from beginning
99 Node* delete_from_beginning(Node* head) {
100     if (!head) return NULL;
101     Node* temp = head;
102     head = head->next;
103     if (head) head->prev = NULL;
104     free(temp);
105     return head;
106 }
107
108 // Delete from end
109 Node* delete_from_end(Node* head) {
110     if (!head) return NULL;
111     if (!head->next) {
112         free(head);
113         return NULL;
114     }
115     Node* temp = head;
116     while (temp->next) {
117         temp = temp->next;
118     }
119     temp->prev->next = NULL;
120     free(temp);
121     return head;
122 }
123
124 // Delete from a specific position
125 Node* delete_from_position(Node* head, int position) {
126     if (position == 1) return delete_from_beginning(head);
127     Node* temp = head;
128     for (int i = 1; i < position && temp; i++) {
129         temp = temp->next;
130     }
131     if (!temp) return head;
132     if (temp->next) temp->next->prev = temp->prev;
133     if (temp->prev) temp->prev->next = temp->next;
134     free(temp);
135     return head;
136 }
137
138 // Count nodes in the list
139 int count_nodes(Node* head) {
140     int count = 0;
141     while (head) {
142         count++;
143         head = head->next;
144     }
145     return count;
146 }
```

```
147
148 // Search an element
149 int search_element(Node* head, int key) {
150     int position = 1;
151     while (head) {
152         if (head->data == key) return position;
153         head = head->next;
154         position++;
155     }
156     return -1;
157 }
158
159 // Reverse the list
160 Node* reverse_list(Node* head) {
161     Node* temp = NULL;
162     Node* current = head;
163     while (current) {
164         temp = current->prev;
165         current->prev = current->next;
166         current->next = temp;
167         current = current->prev;
168     }
169     return temp ? temp->prev : head;
170 }
171
172 // Insert before a given node
173 Node* insert_before(Node* head, Node* target, int data) {
174     if (!target) return head;
175     if (target == head) return insert_at_beginning(head, data);
176     Node* new_node = create_node(data);
177     new_node->prev = target->prev;
178     new_node->next = target;
179     if (target->prev) target->prev->next = new_node;
180     target->prev = new_node;
181     return head;
182 }
183
184 // Insert after a given node
185 Node* insert_after(Node* head, Node* target, int data) {
186     if (!target) return head;
187     Node* new_node = create_node(data);
188     new_node->next = target->next;
189     new_node->prev = target;
190     if (target->next) target->next->prev = new_node;
191     target->next = new_node;
192     return head;
193 }
194
195 // Delete a node by value
196 Node* delete_by_value(Node* head, int value) {
197     Node* temp = head;
```

```
198     while (temp && temp->data != value) {
199         temp = temp->next;
200     }
201     if (!temp) return head;
202     if (temp == head) return delete_from_beginning(head);
203     if (temp->next) temp->next->prev = temp->prev;
204     if (temp->prev) temp->prev->next = temp->next;
205     free(temp);
206     return head;
207 }
208
209 // Sort the list using bubble sort
210 Node* bubble_sort(Node* head) {
211     if (!head) return head;
212     int swapped;
213     Node* temp;
214     Node* last = NULL;
215     do {
216         swapped = 0;
217         temp = head;
218         while (temp->next != last) {
219             if (temp->data > temp->next->data) {
220                 int temp_data = temp->data;
221                 temp->data = temp->next->data;
222                 temp->next->data = temp_data;
223                 swapped = 1;
224             }
225             temp = temp->next;
226         }
227         last = temp;
228     } while (swapped);
229     return head;
230 }
231
232 // Main function to demonstrate all operations
233 int main() {
234     Node* head = NULL;
235
236     // Create and display list
237     printf("Create doubly linked list with 3 nodes:\n");
238     head = create_list(3);
239     display_forward(head);
240
241     // Insert at beginning
242     printf("Insert 10 at beginning:\n");
243     head = insert_at_beginning(head, 10);
244     display_forward(head);
245
246     // Insert at end
247     printf("Insert 50 at end:\n");
248     head = insert_at_end(head, 50);
```

```
249     display_forward(head);
250
251     // Insert at position
252     printf("Insert 25 at position 3:\n");
253     head = insert_at_position(head, 25, 3);
254     display_forward(head);
255
256     // Delete from beginning
257     printf("Delete from beginning:\n");
258     head = delete_from_beginning(head);
259     display_forward(head);
260
261     // Delete from end
262     printf("Delete from end:\n");
263     head = delete_from_end(head);
264     display_forward(head);
265
266     // Delete from position
267     printf("Delete from position 2:\n");
268     head = delete_from_position(head, 2);
269     display_forward(head);
270
271     // Count nodes
272     printf("Number of nodes: %d\n", count_nodes(head));
273
274     // Search for an element
275     printf("Search for element 25:\n");
276     int position = search_element(head, 25);
277     if (position != -1) {
278         printf("Element found at position: %d\n", position);
279     } else {
280         printf("Element not found\n");
281     }
282
283     // Reverse the list
284     printf("Reverse the list:\n");
285     head = reverse_list(head);
286     display_forward(head);
287
288     // Bubble sort the list
289     printf("Sort the list:\n");
290     head = bubble_sort(head);
291     display_forward(head);
292
293     return 0;
294 }
```

OUTPUT -

```
Create doubly linked list with 3 nodes:
Enter data for node 1: 8
Enter data for node 2: 13
Enter data for node 3: 42
8 -> 13 -> 42 -> NULL
Insert 10 at beginning:
10 -> 8 -> 13 -> 42 -> NULL
Insert 50 at end:
10 -> 8 -> 13 -> 42 -> 50 -> NULL
Insert 25 at position 3:
10 -> 8 -> 25 -> 13 -> 42 -> 50 -> NULL
Delete from beginning:
8 -> 25 -> 13 -> 42 -> 50 -> NULL
Delete from end:
8 -> 25 -> 13 -> 42 -> NULL
Delete from position 2:
8 -> 13 -> 42 -> NULL
Number of nodes: 3
Search for element 25:
Element not found
Reverse the list:
42 -> 13 -> 8 -> NULL
Sort the list:
8 -> 13 -> 42 -> NULL
```